

Aula 6 – Polimorfismo em C#

Programação Orientada a Objetos
Professor: Eduardo Luan Pilonetto

Objetivos da Aula

- Compreender o conceito de polimorfismo
- Aplicar sobrescrita de métodos
- Utilizar comportamentos diferentes em objetos
- Trabalhar com métodos virtuais e sobrecarga

O que é Polimorfismo?

- Polimorfismo significa 'muitas formas'
- Objetos podem se comportar de maneiras diferentes
- Permite reutilização e flexibilidade
- Base da Programação Orientada a Objetos

Tipos de Polimorfismo

- Polimorfismo de Sobrescrita (Override)
- Polimorfismo de Sobrecarga (Overload)
- Polimorfismo em tempo de compilação
- Polimorfismo em tempo de execução

Métodos Virtuais

- Utilizamos a palavra-chave virtual
- Permite redefinir métodos na classe filha
- Facilita extensibilidade
- Muito usado em herança

Exemplo de Método Virtual

```
class Animal
{
    public virtual void EmitirSom()
    {
        Console.WriteLine("Som genérico");
    }
}
```

Override – Sobrescrita

- Utilizamos override na classe filha
- Substitui comportamento da classe pai
- Executado em tempo de execução

Exemplo de Override

```
class Cachorro : Animal
{
    public override void EmitirSom()
    {
        Console.WriteLine("Au Au!");
    }
}
```


Outro Exemplo de Override

```
class Gato : Animal
{
    public override void EmitirSom()
    {
        Console.WriteLine("Miau!");
    }
}
```

Testando o Polimorfismo

```
Animal a1 = new Cachorro();
```

```
Animal a2 = new Gato();
```

```
a1.EmitirSom();
```

```
a2.EmitirSom();
```

Saída Esperada

Au Au!

Miau!

Polimorfismo em Coleções

- Podemos armazenar diferentes objetos
- Todos usando a mesma classe base
- Facilita manutenção do código

Exemplo com Lista

```
List<Animal> animais = new List<Animal>();
```

```
animais.Add(new Cachorro());
```

```
animais.Add(new Gato());
```

```
foreach(var animal in animais)
```

```
{
```

```
    animal.EmitirSom();
```

```
}
```

Sobrecarga de Métodos

- Métodos com mesmo nome
- Parâmetros diferentes
- Também chamado de Overload

Exemplo de Sobrecarga

```
class Calculadora
{
    public int Somar(int a, int b)
    {
        return a + b;
    }

    public double Somar(double a, double b)
    {
        return a + b;
    }
}
```

Diferença entre Override e Overload

Override:

- Mesmo método
- Classe filha redefine comportamento

Overload:

- Mesmo nome
- Parâmetros diferentes

Classe Abstrata e Polimorfismo

- Classes abstratas servem como modelo
- Não podem ser instanciadas
- Podem conter métodos abstratos

Exemplo de Classe Abstrata

```
abstract class Funcionario  
{  
    public abstract double CalcularSalario();  
}
```

Exemplo Completo

```
class Gerente : Funcionario
{
    public override double CalcularSalario()
    {
        return 10000;
    }
}
```

Interfaces e Polimorfismo

- Interfaces definem contratos
- Classes implementam comportamentos
- Excelente para desacoplamento

Exemplo de Interface

```
interface IPagamento  
{  
    void Pagar();  
}
```

Implementação da Interface

```
class Pix : IPagamento
{
    public void Pagar()
    {
        Console.WriteLine("Pagamento via PIX");
    }
}
```

Vantagens do Polimorfismo

- Flexibilidade
- Reutilização de código
- Facilidade de manutenção
- Código mais organizado

Boas Práticas

- Utilize herança com moderação
- Prefira interfaces quando possível
- Evite duplicação de código
- Nomeie métodos corretamente

Erros Comuns

- Esquecer virtual na classe pai
- Assinaturas incompatíveis
- Confundir overload com override

Resumo da Aula

- Polimorfismo permite múltiplos comportamentos
- Override redefine métodos
- Overload cria múltiplas versões
- Métodos virtuais permitem extensibilidade

Exercícios

1. Criar classe Animal com método virtual
2. Criar classes derivadas
3. Implementar sobrecarga
4. Criar interface de pagamento
5. Simular sistema de funcionários

Encerramento

Dúvidas?

Próxima aula: Classes Abstratas e Interfaces